ADA Tarea 3

César Augusto Zuluaga Araque

Pontificia Universidad Javeriana Cali

13/04/2026

7. The wildly popular Spanish-language search engine El Goog needs to do a serious amount of computation every time it recompiles its index. Fortunately, the company has at its disposal a single large supercomputer, together with an essentially unlimited supply of high-end PCs.

They've broken the overall computation into n distinct jobs, labeled $J_1, J_2, \ldots, J_n$, which can be performed completely independently of one another. Each job consists of two stages: first it needs to be *preprocessed* on the supercomputer, and then it needs to be *finished* on one of the PCs. Let's say that job J_i needs p_i seconds of time on the supercomputer, followed by f_i seconds of time on a PC.

Since there are at least n PCs available on the premises, the finishing of the jobs can be performed fully in parallel—all the jobs can be processed at the same time. However, the supercomputer can only work on a single job at a time, so the system managers need to work out an order in which to feed the jobs to the supercomputer. As soon as the first job

Figure 1:

in order is done on the supercomputer, it can be handed off to a PC for finishing; at that point in time a second job can be fed to the supercomputer; when the second job is done on the supercomputer, it can proceed to a PC regardless of whether or not the first job is done (since the PCs work in parallel); and so on.

Let's say that a *schedule* is an ordering of the jobs for the supercomputer, and the *completion time* of the schedule is the earliest time at which all jobs will have finished processing on the PCs. This is an important quantity to minimize, since it determines how rapidly El Goog can generate a new index.

Give a polynomial-time algorithm that finds a schedule with as small a completion time as possible.

Figure 2:

Punto 1

0.1 Entradas y Salidas

Entrada:
Se recibe un arreglo A[0..N) el cual contiene parejas de números enteros positivos de la forma (p_i, f_i) tal que p_i indica el tiempo que tiene que pasar el proceso i en el supercomputador y f_i es el tiempo que tarda el proceso i para el proceso de finishing y estas parejas nos referiremos como trabajos.

Salida:
El minimo tiempo de finalización posible despúes de que todos los procesos de A hayan sido procesados en el supercomputador y finalizados en los computadores individuales.

0.2 Explicación del algoritmo:

El algoritmo que se propone utiliza la siguiente euristica de programación voraz, la cual consiste en ordenar el arreglo A en orden descendente con respecto al valor f de las parejas, de tal forma que el arreglo A cumple que el valor F de las parejas cumple para todo el arreglo que $f_0 \geq f_1 \geq f_2 \geq ... \geq f_{n-1}$, una vez con el arreglo de esta forma , elegimos el primer valor que no haya sido procesado en el orden especificado arriba.

Una vez con este valor tenemos que hacer la siguiente operación, al tiempo en el que podemos procesar el siguiente elemento en el supercomputador sera igual a el tiempo transcurrido total del supercomputador más el tiempo p de procesamiento que estemos viendo, y el tiempo de finalización de todos nuestros elementos sera el maximo entre el tiempo de finalización hasta ese momento (es decir el tiempo maximo de finalización entre los procesos que ya hemos procesado) y el tiempo total más el tiempo en cpu y sumado el tiempo en los computadores individuales. Al hacer esto para los N procesos que tiene nuestro arreglo nos dara el resultado correcto.

0.3 Estrategia Voraz:

Ordenar las parejas del arreglo A con respecto al valor f en orden descendente tal que $f_0 \geq f_1 \geq f_2 \geq ... \geq f_{n-1}$ y procesarlos en ese orden.

0.4 Correctitud:

Teorema de optimización local:
Sea el arreglo A ordenado de forma descendente con respecto al valor f_i procesamos los trabajos de A en ese orden y sea $a_m = (p_m, f_m)$ el trabajo con mayor tiempo de finalización es decir que cumple la siguiente propiedad

$$f_m \geq f_i \quad \forall\, i \in A$$

Existe un orden óptimo de procesamiento que comienza con a_m.

Demostración:

Sea B un orden optimo para procesar los trabajos de A entonces tenemos dos casos:

1. Si el trabajo a_m se procesa primero en B, entonces el orden coincide con la elección voraz y no hay nada que demostrar.

2. El trabajo a_m no se procesa primero en B, por lo tanto existe un trabajo $b_j = (p_j, f_j)$ que se procesa primero que el trabajo a_m en el orden que establece B y el trabajo a_m tiene una posición arbitraria en B entonces tiene que existe un orden B' que ejecute el trabajo a_m antes que el trabajo b_j y es optimo.

Sea T el tiempo acumulado antes de ejecutar el trabajo b_j según el orden B

$$T = \sum_{q=0}^{j-1} p_q$$

Sea además H la suma de los tiempos de procesamiento de todos los trabajos entre b_j y a_m en el orden B.

Luego de ejecutar b_j el tiempo acumulado sera igual a $T + p_j$.El trabajo a_m no puede comenzar antes de $T + p_j + H$, por lo que el tiempo de finalización para el trabajo a_m es:

$T+p_j+H+p_m+f_m$. Por su parte, el tiempo de finalización de b_j es $T+p_j+f_j$.

Para la comparación entre B y B' basta considerar los trabajos b_j y a_m. Los demás trabajos se ejecutan en el mismo orden en ambos arreglos, por lo que sus tiempos de finalización no cambian y no afectan el valor máximo.

Si el tiempo de finalización dependiera de alguno de esos trabajos, el intercambio sería irrelevante. En el caso contrario, adelantar a_m no empeora el tiempo máximo total, pues el valor resultante es menor o igual al obtenido en B.

Por lo tanto, el tiempo máximo de finalización del orden B está dado por

$$max(T + p_j + H + p_m + f_m, T + p_j + f_j)$$

Como sabemos que $f_m \geq f_j$, $p_j \geq 0$ y que $H \geq 0$ entonces:

$$T + p_j + H + p_m + f_m \geq T + p_j + f_j$$

Entonces el tiempo de finalización en B es:

$$T + p_j + H + p_m + f_m$$

Ahora consideremos el orden B' el cual intercambia el orden de ejecución del trabajo b_j con el trabajo a_m sin afectar el orden de ejecución de los otros

trabajos.

En este orden B', el calculo del siguiente tiempo acumulado ahora es $T+p_m$, y el tiempo de finalización del trabajo b_j ahora es $T+p_m+H+p_j+f_j$

Por lo tanto, el tiempo máximo de finalización en el orden B' está acotado por:

$$\max\big(T+p_m+f_m,\ T+p_m+H+p_j+f_j\big).$$

Nosotros no sabemos cual de estos dos es mayor, pero sabemos que se tiene que cumplir la siguiente propiedad para que nos de un optimo

$$\max(T+p_m+f_m, T+H+p_m+p_j+f_j) \leq T+p_j+H+p_m+f_m$$

Así que primero tomemos lo que esta a la izquierda de la operación max.

La premisa sera:

$$T+p_m+f_m \leq T+p_j+H+p_m+f_m$$

Esto es verdad trivialmente pues establecimos que $p_j \geq 0$ y que $H \geq 0$ por lo tanto suponiendo que tomen el menor valor posible, siendo este 0 nos queda lo siguiente

$$T+p_m+f_m \leq T+p_j+H+p_m+f_m$$

$$T+p_m+f_m \leq T+0+0+p_m+f_m$$

$$T+p_m+f_m \leq T+p_m+f_m$$

Esto es exactamente igual por lo tanto $T+p_m+f_m$ es menor o igual que el resultado optimo de B.

Nuestra otra premisa es

$$T+p_m+H+p_j+f_j \leq T+p_j+H+p_m+f_m$$

Aqui el unico valor diferente es f_m y f_j y como previamente ya habiamos establecido que $f_m \geq f_j$ entonces nuestro enunciado es verdadero.

Como ya hemos establecido que sin importar el valor que tome nuestro orden sera menor o igual que el resultado optimo sin escoger primero a_m y que el resultado de B sin escoger primero a_m es un resultado optimo entonces podemos concluir que Por lo tanto,

$$\max\big(T+p_m+f_m,\ T+p_m+p_j+f_j\big) \leq T+p_j+p_m+f_m.$$

Asi que procesar el trabajo a_m primero que el trabajo b_j no empeora el optimo y como establecimos que el orden B es optimo entonces el resultado de procesar los trabajos en el orden B' ó el orden B nos dara el mismo resultado optimo.

0.5 Función Objetivo:

$$0 \leq n \leq N, \quad 0 \leq t, \quad 0 \leq fin.$$

$\phi(n,t)$ Calcula el tiempo de finalización minimo en el cual acaban todos los trabajos restante del arreglo que va desde A[n..N), teniendo en cuenta que en el tiempo t es cuando se puede empezar a poner cosas en el Super computador.

0.6 Reformulación de la salida

$\phi(0,0)$

0.7 Planteamiento Recurrente

Sea $A[n] = (p_n, f_n)$. La recurrencia se define como:

$$\phi(n,t) = \begin{cases} t, & \text{si } n = N \\ \max\big(t + p_n + f_n,\ \phi(n+1, t+p_n)\big), & \text{si } n < N \end{cases}$$

donde:

- $t + p_n$ es el tiempo en el que el supercomputador termina el trabajo n,
- $t + p_n + f_n$ es el tiempo en el que el trabajo n termina completamente,
- $\phi(n+1, t+p_n)$ calcula el óptimo para los trabajos restantes.

0.8 Teorema de Optimización Global:

Sea $0 \leq n \leq N$ y $0 \leq t$ la función $\phi(n,t)$ calcula correctamente el tiempo de finalización de los trabajos restantes en A[n..N)

Demostración:
Se procede por inducción sobre el tamaño de la variable k = N - n, en donde k representa la cantidad de trabajas restantes.

Caso Base:
k = 0. En este caso como tenemos

$$k = N - n$$

$$0 = N - n$$

$$N = n$$

Esto indica que el arreglo va de A[N..N) lo cual es un intervalo vacío, por lo tanto la solución es t, puesto que al no haber más trabajos que procesar, se devuelve el tiempo acumulado hasta ese momento.

Hipótesis inductiva:
Para cualquier subarreglo restante de longitud $k' < k$, es decir, para cualquier $n' > n$ y $t' \geq t$, la función $\phi(n', t')$ calcula correctamente el tiempo de finalización mínimo de los trabajos en $A[n'..N)$.

Caso inductivo:
Para cualquier n distinto de N se hace el siguiente llamado

$$\max\big(t + p_n + f_n,\ \phi(n+1, t+p_n)\big)$$

Por la hipótesis inductiva, el llamado recursivo $\phi(n+1, t+p_n)$ calcula correctamente el tiempo mínimo de finalización de los trabajos restantes. Al tomar el máximo con $t + p_n + f_n$, nos aseguramos de considerar el tiempo de finalización del trabajo actual y de los trabajos restantes, garantizando así que se capture correctamente el tiempo de finalización total mínimo.

Conclusión: El algoritmo calcula correctamente el tiempo de finalización minimo para los trabajos de A[n..N) con la función $\phi(n, t)$.

Corolario: El llamado a $\phi(0, 0)$ calcula corretamente el tiempode finalización para los trabajos desde A[0..N) por la demostración anterior.

0.9 Complejidad temporal:

A[n][0] = p_n A[n][1] = f_n

Algorithm 1 El Goog

```
function SORT(A)
function φ(n, t):
    if n == len(A)
        ans = t
    else
        max(t + A[n][0] + A[n][1], φ(n + 1, t + A[n][0])
    return ans
```

Aqui si utilizamos la función merge sort que nos organiza el arreglo A en base a nuestro criterio se nos tarda un tiempo total de $Nlog(N)$ y nuestra función phi se demora N pues esta hace un llamado recursivo que siempre incrementa en 1 y acaba cuando llega al final del arreglo A que tiene tamaño N y las operaciónes que hacemos dentro de este se hacen en tiempo O(1) ignorando el llamado recursivo, por lo tanto la complejidad de nuestro algoritmo es:

$$T(N) = N\log(N) + N$$
$$T(N) = N\log(N)$$
$$T(N) \in O(N\log(N)$$

Punto 2

22. A string w of parentheses (and) is ***balanced*** if it satisfies one of the following conditions:

- w is the empty string.
- $w = (x)$ for some balanced string x
- $w = xy$ for some balanced strings x and y

For example, the string

$$w = ((())()()) (()()) ()$$

is balanced, because $w = xy$, where

$$x = ((())()()) \qquad \text{and} \qquad y = (()()) ().$$

(a) Describe and analyze an algorithm to determine whether a given string of parentheses is balanced.

(b) Describe and analyze a greedy algorithm to compute the length of a longest balanced subsequence of a given string of parentheses. As usual, don't forget to prove your algorithm is correct.

For both problems, your input is an array $w[1..n]$, where for each i, either $w[i] = ($ or $w[i] =)$. Both of your algorithms should run in $O(n)$ time.

Figure 3:

Punto a

0.10 Entradas y salidas

Entrada:
Un arreglo w[1..n] tal que para cualquier $1 \leq i \leq n$, w[i] = '(' ó w[i] = ')'.

Salida:
Un booleano que indique si el arreglo w esta balanceado según los criterios dados en el problema.

0.11 Descripción del algoritmo

Para la resolución del problema se plantea la siguiente solución. Se tiene un valor k, que representa la cantidad de parentesis abiertos hasta el momento, es decir que cada vez que encontremos un parentesis abierto '(' le sumamos 1 a k y cada vez que encontremos un parentesís cerrado ')' y que el valor de k sea mayor que 0 significa que tenemos una pareja de parentesis balanceada y restamos 1 a k, ahora, si encontramos un parentesís cerrado y k es 0, significa que no tiene pareja y por lo tanto nuestro arreglo w no esta balanceado, también el otro caso

que nos indica si nuestro arreglo esta desbalanceado es que una vez recorramos todo el arreglo el valor de k sea mayor que 0, esto significa que hay un parentesís abierto que no pudimos cerrar y por lo tanto no es balanceado w.

0.12 Planteamiento de la funció objetivo

$$1 \leq i \leq n+1$$

$$0 \leq k$$

$\phi(i,k)$, la cual retorna **TRUE** si y solo si la subcadena $w[i..n]$ puede ser balanceada correctamente suponiendo que existen k paréntesis abiertos pendientes de cerrar antes de procesar la posición i. En caso contrario, retorna **FALSE**.

0.13 Reformulación de la Salida:

$\phi(1,0)$

0.14 Planteamiento Recurrente

$$\phi(i,k) = \begin{cases} k == 0, \text{ si } i = n+1 \\ FALSE, \text{ si } w[i] = ')' \text{ and } k = 0 \\ \phi(i+1, k+1), \text{ si } w[i] = '(' \\ \phi(i+1, k-1), \text{ si } w[i] = ')' \text{ and } k > 0 \end{cases}$$

0.15 Correctitud:

Teorema:
El llamado a $\phi(i,k)$ determina correctamente si con los parentesís sin procesar del arreglo desde w[i..n] es balanceado si se tienen k parentesis abiertos.

Demostración:
Se procede por inducción sobre l en donde l = n+1-i y donde l representa el largo de lo que queda por procesar en w.
Casos base:

$l = 0$, lo que implica que $i = n+1$. En este caso ya se han procesado todos los caracteres del arreglo.

- Si $k = 0$, no quedan paréntesis abiertos sin cerrar, por lo tanto la cadena está balanceada y $\phi(i,k)$ retorna **TRUE**.
- Si $k > 0$, existen paréntesis de apertura sin cerrar, por lo tanto la cadena no está balanceada y $\phi(i,k)$ retorna **FALSE**.

Hipotesis inductiva Para un $l' < l$ tal que $l' = n+1-i'$ y $i' > i$, se cumple que el llamado a $\phi(i',k')$ determina correctamente que el arreglo w[i'..n]

esta balanceado si se tienen k' parentesis abierto tal que $k' \neq k$.

Paso inductivo:
Consideremos el llamado $\phi(i, k)$ con $i \leq n$. Existen dos casos según el valor de $w[i]$:

- Si $w[i] = ($, el algoritmo incrementa el número de paréntesis abiertos pendientes y evalúa $\phi(i+1, k+1)$. Por hipótesis inductiva, este llamado es correcto, por lo que $\phi(i, k)$ también lo es.
- Si $w[i] =)$:
  - Si $k = 0$, no existe un paréntesis de apertura que empareje a $w[i]$, por lo que la cadena no puede ser balanceada y $\phi(i, k)$ retorna **FALSE**.
  - Si $k > 0$, el algoritmo empareja este paréntesis con uno previo y evalúa $\phi(i+1, k-1)$. Por hipótesis inductiva, este llamado es correcto, por lo que $\phi(i, k)$ también lo es.

En todos los casos, la función $\phi(i, k)$ retorna el valor correcto. Por lo tanto, por inducción sobre l, la función es correcta para todo $1 \leq i \leq n+1$ y $k \geq 0$.

Corolario: El llamado a $\phi(1, 0)$ evalua correctamente que el arreglo de w[1..n] esta balanceado, por la demostración previa.

0.16 Complejidad Temporal:

Algorithm 2 balanced parenthesis

```
function φ(i, k):
    if i == n+1 :
        ans = k == 0
    else if w[i] ==')' and k > 0
        ans = phi(i + 1, k − 1)
    else if w[i] ==')' and k == 0
        ans = FALSE
    else if w[i] ==' ('
        ans = φ(i + 1, k + 1)
    return ans
```

El algoritmo en el peor de los casos hace n llamados recursivos lo cual seria el tamaño del arreglo, mientras tanto lo que se encuentra dentro de los llamados recursivos tiene complejidad O(1) sin tener encuenta los llamados recursivos por lo tanto la complejidad es de:

$$T(n) = n$$

$$T(n) \in O(n)$$

Punto b:

0.17 Entradas y salidas

Entrada:
Un arreglo w[1..n] tal que para cualquier $1 \leq i \leq n$, w[i] = '(' ó w[i] = ')'.

Salida:
La longitud de la subcadena balanceada más larga perteneciente a w[1..n]

0.18 Estrategia Voraz:

Cada vez que encuentre un parentesis Cerrado ')' y ya he abierto como minimo 1 parentesis '(' entonces emparejo el ultimo parentesís abierto con el parentesís cerrado que encontre.

0.19 Correctitud:

Teorema de Optimización Local:
Sea i una posición en w tal que w[i] = ')' y un contador $k > 0$ tal que k representa la cantidad de parentesis abiertos sin pareja desde w[1..i-1]. Emparejar el parentesis en la posición i con el parentesis que abre que no tiene pareja más cercana a la izquierda no empeora la longitud de la subsequencia balanceada mas larga.

Demostración:
Sea S una subsecuencia balanceada de longitud máxima construida a partir de w, y sea T el conjunto de parejas de paréntesis que definen a S.

Sea $(j, i) \in T$ una pareja tal que $w[j] =' ($' y $w[i] =')'$, con $j < i$. Tenemos dos casos

- La pareja (j,i) cumple con nuestro criterio voraz pues se cumple que no existe un índice q tal que:

  $$j < q < i, \quad w[q] =' ('\text{,} \quad \text{y } q \text{ no está emparejado en } T.$$

  En este caso, el paréntesis $w[j]$ es el paréntesis abierto más cercano disponible a $w[i]$, por lo que la pareja (j, i) coincide con la elección voraz y no es necesario realizar ninguna modificación.

- No cumple nuestro criterio si existe un índice q tal que:

  $$j < q < i, \quad w[q] =' ('\text{,} \quad \text{y } q \text{ no está emparejado en } T.$$

  En este caso, consideremos el conjunto de emparejamientos

  $$T' = (T \setminus \{(j, i)\}) \cup \{(q, i)\}.$$

  Ahora podemos observar lo siguiente:

- El emparejamiento (q, i) es válido, ya que $w[q] =' ('$, $w[i] =')'$ y $q < i$.
- El conjunto T' define una subsecuencia balanceada de la misma longitud que la nos da T, puesto que el tamaño de T' es igual al tamño de T ya que el emparejamiento (q, i) sustituye a (j, i) sin alterar el resto de las parejas.
- El intercambio no afecta a los demás emparejamientos de T, ya que el índice q no pertenecía a ninguna pareja en T y el índice j queda libre sin invalidar ningún emparejamiento existente.

Además, al emparejar el paréntesis cerrado $w[i]$ con el paréntesis abierto más cercano disponible, se preservan paréntesis abiertos ubicados más a la izquierda, los cuales pueden ser utilizados para emparejar paréntesis cerrados posteriores. Por lo tanto, este intercambio no reduce las posibilidades de emparejamiento futuro.

Concluimos que existe una subsecuencia balanceada óptima que cumple el criterio voraz en la posición i.

0.20 Planteamiento Función Objetivo:

$$1 \leq i \leq n + 1$$

$$0 \leq k$$

$\phi(i, k)$ calcula la longitud de la subcadena balanceada más larga en w[i..n] suponiendo que se tienen k parentesis abiertos que no han sido emparejados.

0.21 Reformulación de la salida:

$\phi(1, 0)$

0.22 Planteamiento Recurrente:

$$\phi(i, k) = \begin{cases} 0, \text{ si } i = n + 1 \\ 2 + \phi(i + 1, k - 1), \text{ si } w[i] ==')' \text{ and } k > 0 \\ \phi(i + 1, k + 1), \textbf{ si } w[i] ==' (' \\ \phi(i + 1, k), \textbf{ si } w[i] ==')' \text{ and } k = 0 \end{cases}$$

0.23 Justificación de la función objetivo sin el conjunto T

En la formulación recursiva de $\phi(i, k)$ no se tiene el conjunto T explicitamente, ya que toda la información relevante para tomar decisiones óptimas locales está contenida en los parámetros (i, k):

- i indica la posición actual en el arreglo w, por lo que sabemos qué paréntesis estamos considerando.

- k representa la cantidad de paréntesis abiertos sin pareja hasta la posición $i-1$, es decir, los candidatos disponibles para emparejar el paréntesis cerrado en $w[i]$ si existe.

Dado que el criterio voraz local garantiza que siempre se puede emparejar el paréntesis cerrado con el paréntesis abierto más cercano disponible sin afectar la longitud máxima de la subsecuencia balanceada, no es necesario tener que parentesis ya hemos emparejado. La recursión sobre (i, k) captura implícitamente todos los posibles emparejamientos válidos y calcula correctamente la longitud máxima de la subsecuencia balanceada.

0.24 Correctitud:

Teorema: El llamado a $\phi(i, k)$ calcula correctamente el tamaño de la subcadena balanceada más larga posible en el arreglo w[i..n] suponiendo que se tienen k parentesis abiertos sin emparejar.

Demostración:
Procedemos por inducción sobre l en donde l = n+1-i y l representa la cantidad de parentesis restantes en el arreglo
w. **Caso Base:** l = 0, es decir i = n+1, esto indica que ya llegamos al final del arreglo, como ya no hay más parentisis que podamos emparejar para extender el tamaño de la subcadena más larga balanceada, entonces devolvemos 0.

Hipotesis inductiva: Supongamos un valor $l' < l$ tal que l' = n+1-i' y $i' > i$ el llamado a la función $\phi(i', k')$, calcula correctamente el tamaño para la subsequencia balanceada más grande posible en el arreglo w[i'..n] suponiendo que se tienen k' parentesis abiertos.

Paso Inductivo: En nuestro algoritmo tenemos 3 llamados recursivos diferentes, por lo que vamos a ir uno por uno:

- **Caso 1:** $w[i] =')'$ y $k > 0$. En este caso, existe al menos un paréntesis abierto disponible para emparejar. Emparejarlo produce una ganancia inmediata de 2 unidades en la longitud de la subsecuencia balanceada. Cualquier solución óptima debe realizar este emparejamiento, ya que ignorarlo no puede permitir formar más parejas en el futuro. Por lo tanto, el valor óptimo es:
$$2 + \phi(i+1, k-1)$$
El llamado recursivo es correcto por hipótesis inductiva.

- $\phi(i+1, k+1)$**, si** $w[i] =='('$ en este caso, el llamado es correcto por hipotesis inductiva, pues al encontrarnos un parentesis abierto esto significa que tenemos otro candidato para las parejas, por ende al sumarlo y hacer el llamado recursivo se obtiene correctamente el resultado esperado.

- $\phi(i+1, k)$**, si** $w[i] =='('$ and $k = 0$, este llamado siginifica que no teniamos un parentesis abierto para emparejar con el ')' por ende este lo ignoramos

ya que sabemos que no hace parte de una solución optima pues no tenemos con que emparejarlo para crear una solucíon balanceada que lo contenga, por ende nuestro llamado es correcto por Hipotesis inductiva.

Por lo tanto, los llamados a $\phi(i, k)$ son correctos, pues en cada iteración el algoritmo toma una decisión local óptima y reduce el problema a un subproblema de menor tamaño, el cual se resuelve correctamente por la hipótesis inductiva.

Corolario:
El llamado a la función $\phi(1, 0)$ calcula correctamente el tamaño de la subsequencia balanceada más larga de w[1..n] según la demostración de la correctitud del algoritmo

0.25 Complejidad Temporal:

Algorithm 3 Longest balanced subsequence

```
function φ(i, k):
    if i == n+1 :
        ans = 0
    else if w[i] ==')' and k > 0
        ans = 2 + phi(i + 1, k − 1)
    else if w[i] ==')' and k == 0
        ans = phi(i + 1, k)
    else if w[i] ==' ('
        ans = φ(i + 1, k + 1)
    return ans
```

La función $\phi(i, k)$ realiza a lo sumo un llamado recursivo en cada ejecución, y en dicho llamado el índice i se incrementa en una unidad. Por lo tanto, la profundidad máxima de la recursión es n, correspondiente al recorrido completo del arreglo w., a parte de los llamados recursivos podemos evidenciar que todos los procesos que se ejecutan dentro de la función son comparaciones y asignaciones los cuales tienen una complejidad de O(1) por lo tanto, la complidad del algoritmo es de:

$$T(n) \in O(n)$$

En donde n es el tamaño del arreglo w.